

对顶堆

```
#include <bits/stdc++.h>
using namespace std;
#define int long long
#define inf 0x3f3f3f3f3f3f3f
#define pii pair<int, int>
#define endl '\n'
const int N=1e6+10;
const int N1=1e3+10;
const int mod=1e9+7;
const int element=3;
int a[N];
vector<int> G[N];
multiset<int> q1;
multiset<int,greater<int>> q2;
int len,lsum,rsum;
void restruct(){
    int k=(len+1)/2;//默认偶数时中位数是中间偏左的。
    while(q1.size()>k){
        lsum-=*q1.rbegin();
        rsum+=*q1.rbegin();
        q2.insert(*q1.rbegin()),q1.erase(--q1.rbegin().base());
    }
    while(q1.size()<k){
        lsum+=*q2.rbegin();
        rsum-=*q2.rbegin();
        q1.insert(*q2.rbegin()),q2.erase(--q2.rbegin().base());
    }
}
void insert(int x){
    len++;
    if(q1.empty()||x<*q1.rbegin()){
        lsum+=x;
        q1.insert(x);
    }
    else{
        rsum+=x;
        q2.insert(x);
    }
    restruct();
}
```

```

int qmid(){
    if(len==0) return 0;
    return *q1.rbegin();
}

void del(int x){
    len--;
    auto l=q1.lower_bound(x);
    auto r=q2.lower_bound(x);
    if(l!=q1.end()){
        lsum-=x;
        q1.erase(l);
    }else if(r!=q2.end()){
        rsum-=x;
        q2.erase(r);
    }else{
        assert(0);
    }
    reconstruct();
}

void clear(){
    len=rsum=lsum=0;
    q1.clear();
    q2.clear();
}

int cal(){
    if(len==0) return 0;
    int cntl=(len+1)/2, cntr=len-(len+1)/2;
    int mid=qmid();
    return mid*cntl-lsum+rsum-mid*cntr;
}

void Atomatic_AC_machine(){
    int n,k;cin>>n>>k;
    vector<int> a(n+1);
    for(int i=1;i<=n;i++){
        cin>>a[i];
    }
    for(int i=1;i<=n;i++){
        a[i]-=i;
    }
    int l=1;
    int ans=0;
    for(int r=1;r<=n;r++){
        insert(a[r]);
    }
}

```

```

    while(cal()>k){
        // cout<<l<<" "<<r<<" "<<cal()<<" "<<qmid()<<" "<<lsum<<" "<<rsum<<endl;
        del(a[l]);
        l++;
    }
    // cout<<l<<" "<<r<<" "<<cal()<<" "<<qmid()<<" "<<lsum<<" "<<rsum<<endl;
    ans=max(r-l+1,ans);
}
cout<<ans<<endl;
clear();
}
signed main(){
    ios::sync_with_stdio(false);cin.tie(0);cout.tie(0);
    int t=1;
    cin>>t;
    while(t--){
        Atomic_AC_machine();
    }
    return 0;
}

```

bitset

```
#include <bits/stdc++.h>
using namespace std;
#define int long long
#define endl "\n"
const int M=61000;
const int N=301;
#pragma GCC optimize(3,"Ofast","inline")
bitset<N> ch[M][26]; //每个bitset表示第i位第j个字母第k个字符串是否和j不同
bitset<N> A,C;
int ans[N];
int n,q,m,K;
signed main(){
    ios::sync_with_stdio(false);
    cin.tie(0);cout.tie(0);
    cin>>n>>q>>m>>K;
    for(int i=1;i<=n;i++){
        string s;cin>>s;
        s=" "+s;
        for(int j=1;j<=m;j++){
            for(int k=0;k<26;k++){
                if(s[j]-'a'!=k) ch[j][k].set(i); //不同
            }
        }
    }
    // cout<<ch[1]['k'-'a']<<endl;
    for(int i=1;i<=q;i++){
        string s;cin>>s;
        s=" "+s;
        memset(ans,0,sizeof(ans));
        for(int j=1;j<=n;j++) A.set(j);
        for(int j=1;j<=m;j++){
            C=A&ch[j][s[j]-'a'];
            for(auto k=C._Find_first();k!=C.size();k=C._Find_next(k)){
                if((++ans[k]>K)) A[k]=0;
            }
        }
        cout<<A.count()<<endl;
    }
    return 0;
}
```